

# **Let the Flows Tell: Solving Graph Combinatorial Optimization Problems with GFlowNets**

**NeurIPS 2023**

**박민제\*, 김윤하, 백승언, 오영화**

# Contents

## 01 Background

- Combinatorial Optimization
- Generative Flow Network (GFlowNets)

## 02 Methods

- MDP Design for GFlowNet
- Solving Training Inefficiency

## 03 Experiments

## 04 Conclusion

## ▪ GFlowNet 기반 Graph Combinatorial Optimization Solver 제안

- Combinatorial optimization(CO)을 풀기 위해 machine learning을 활용하려는 연구<sup>1</sup>가 활발히 진행 중.
- CO solver에 잘 부합하는 특성을 가진, GFlowNet<sup>2</sup>이라는 새로운 생성형 알고리즘이 최근에 제안되었음.
- 본 논문에서는 graph에서의 NP-hard CO를 풀기 위한 GFlowNet 방법론을 새롭게 제안함.
  - GFlowNet 기반 CO 풀이를 위한 Markov decision process (MDP) 모델링.
  - GFlowNet을 단순 적용했을 때 발생하는 문제점 확인 및 이를 해결하기 위한 방법론 제시.

1. Bengio, Y., Lodi, A., & Prouvost, A. (2021). Machine learning for combinatorial optimization: a methodological tour d'horizon. *European Journal of Operational Research*, 290(2), 405-421.

2. Bengio, E., Jain, M., Korablyov, M., Precup, D., & Bengio, Y. (2021). Flow network based generative models for non-iterative diverse candidate generation. *Advances in Neural Information Processing Systems*, 34, 27381-27394.

# **01 Background**

# Background – Combinatorial Optimization

## ■ Combinatorial Optimization (CO)<sup>1</sup>

- Discrete solution space에서의 최적화 문제

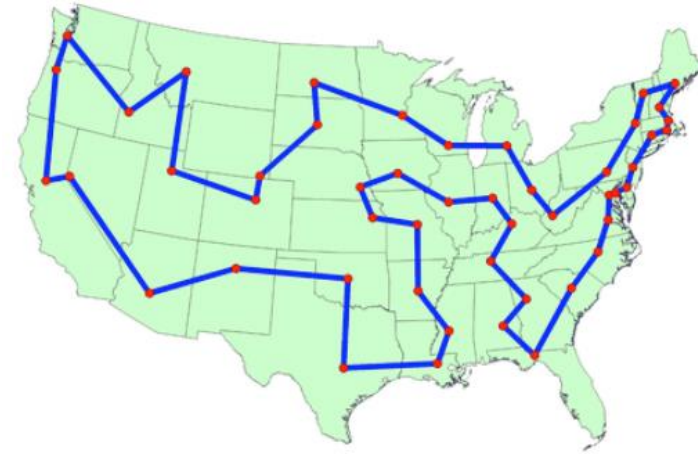
$$\min_{x \in \mathcal{X}_f} C(x)$$

- Solution  $x \in \mathcal{X} = \{0,1\}^N$
- Feasible solution set  $\mathcal{X}_f$
- Cost  $R: x \rightarrow \mathbb{R}^+$

- E.g., Traveling salesman problem

–  $x_{i,j}$ : 도시  $i$ 와  $j$ 를 연결 /  $\mathcal{X}_f$ : 모든 도시를 단 한 번만 방문 /  $C(x) = \sum_{i,j} d_{i,j} x_{i,j}$ : 거리의 총 합.

- 대부분 NP-hard 문제이며, 따라서 polynomial time 내에 해를 찾는 것은 intractable.



1. Bengio, Y., Lodi, A., & Prouvost, A. (2021). Machine learning for combinatorial optimization: a methodological tour d'horizon. *European Journal of Operational Research*, 290(2), 405-421.

# Background – Combinatorial Optimization

## ▪ CO with Machine Learning Models<sup>1</sup>

- 데이터 기반 CO solver 모델.
- **Objective:** 비교적 빠른 시간 안에 near-optimal solution을 찾는 것.
- **방법론**
  - **Solver 기반 지도 학습**
    - Numerical solver를 통해 획득한 optimal/sub-optimal solution을 맞추도록 학습<sup>2</sup>.
    - Solution을 얻기 쉽지 않다.
  - **강화학습**
    - 단계적으로 solution을 찾도록 MDP 구성 및 policy 학습<sup>3,4</sup>.
    - Mode-collapse issue: 다양한 해를 찾지 못하고 하나의 최적해만을 추구하도록 학습되는 경향.
  - **생성모델 학습**
    - Diffusion 기반 모델<sup>5</sup>의 경우, 해를 찾는 데 시간이 비교적 오래걸리는 경향.

1. Bengio, Y., Lodi, A., & Prouvost, A. (2021). Machine learning for combinatorial optimization: a methodological tour d'horizon. *European Journal of Operational Research*, 290(2), 405-421.

2. Gasse, M., Chételat, D., Ferroni, N., Charlin, L., & Lodi, A. (2019). Exact combinatorial optimization with graph convolutional neural networks. *Advances in neural information processing systems*, 32.

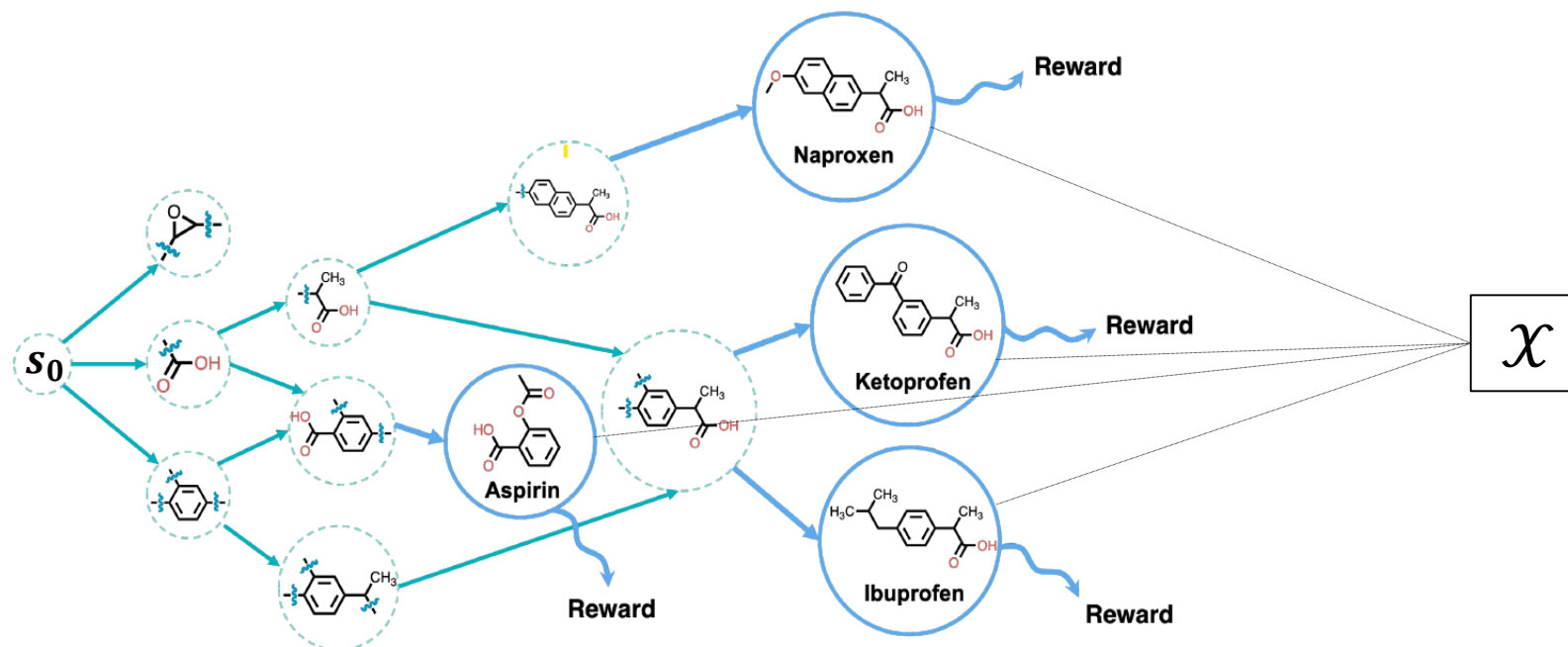
3. Khalil, E., Dai, H., Zhang, Y., Dilkina, B., & Song, L. (2017). Learning combinatorial optimization algorithms over graphs. *Advances in neural information processing systems*, 30.

4. Chen, X., & Tian, Y. (2019). Learning to perform local rewriting for combinatorial optimization. *Advances in neural information processing systems*, 32.

5. Sun, Z., & Yang, Y. (2023). Difusco: Graph-based diffusion solvers for combinatorial optimization. *Advances in Neural Information Processing Systems*, 36, 3706-3731.

# Background – Generative Flow Networks (GFlowNets)

## ▪ GFlowNet Overview<sup>1</sup>

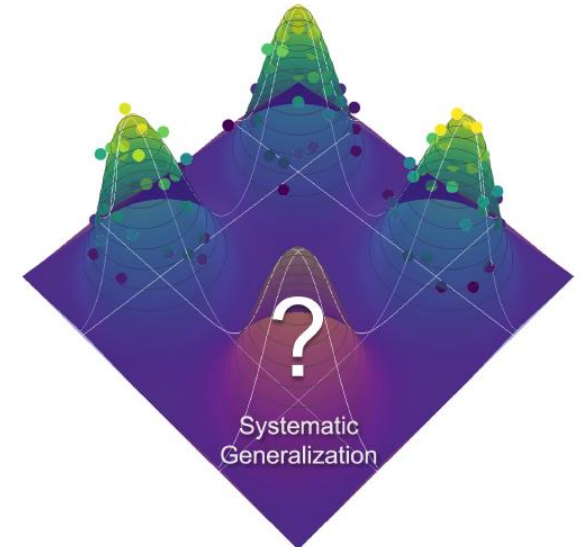
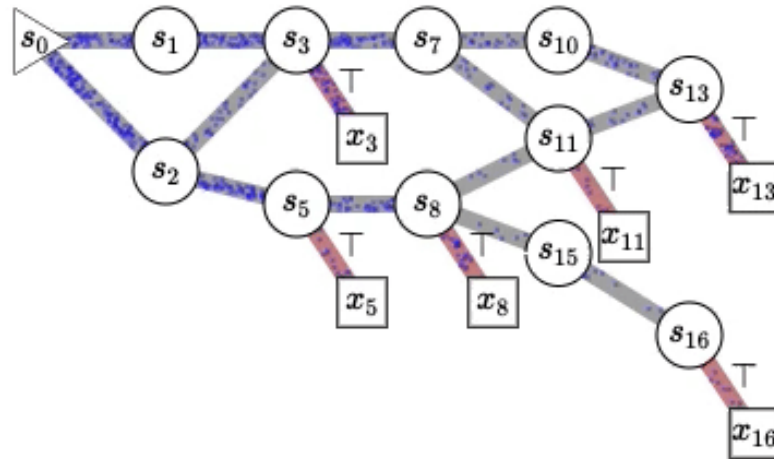


- $\mathcal{S}$ : a set of discrete states.
- $\mathcal{A}$ : a set of actions,  $\mathcal{A} \subseteq \mathcal{S} \times \mathcal{S}$ .
- $P_F(s'|s)$ : a policy,  $s \in \mathcal{S} \setminus \mathcal{X}$ .
- $P_F(\tau) = \prod_{i=0}^{n-1} P_F(s_{i+1}|s_i)$ .
- $\mathcal{X}$ : a set of terminal states,  $\mathcal{X} \subset \mathcal{S}$ .
- $\tau = (s_0, \dots, s_n)$ : a complete trajectory,  $s_n \in \mathcal{X}$ .
- $R(x)$ : a reward function.
- $P_F^\top(x) = \sum_{\tau \rightarrow x} P_F(\tau)$ .

# Background – Generative Flow Networks (GFlowNets)

## ▪ GFlowNet Overview<sup>1</sup>

- **Objective:** Fit  $P_F^\top(x)$  to be proportional to  $R(x)$ .
  - $P_F^\top(x) \propto F(x) = R(x)$  ( $F(x)$ : flow, unnormalized probability.)
  - Multimodal reward function  $R(x)$ 를 가정.
  - 하나의 네트워크로, reward를 최대화하는 solution을 다양하게 생성할 수 있는 generative model 학습.
  - Because of the sequential construction of objects  $x$  (with an arbitrary number of steps), it is **very convenient to generate variable-size structured objects**, like sets, graphs, lists, programs or other recursively constructed data structures.



1. Jain, M., Deleu, T., Hartford, J., Liu, C. H., Hernandez-Garcia, A., & Bengio, Y. (2023). Gflownets for ai-driven scientific discovery. Digital Discovery, 2(3), 557-577.



# Background – Generative Flow Networks (GFlowNets)

## ▪ GFlowNet Training

- **Trajectory Balance (TB)**<sup>1</sup>: Forward flow = backward flow for a  $\tau$ .

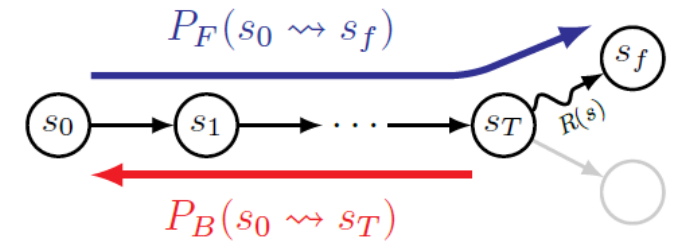
- Target network: forward policy  $P_F(s'|s; \theta)$
- Auxiliary network: backward policy  $P_B(s|s'; \theta)$
- TB loss

$$\ell_{TB}(\tau; \theta) = \log \frac{Z_\theta \prod_{i=0}^{n-1} P_F(s_{i+1}|s_i; \theta)}{R(x) \prod_{i=0}^{n-1} P_B(s_i|s_{i+1}; \theta)} \quad (Z_\theta: \text{learnable scalar value})$$

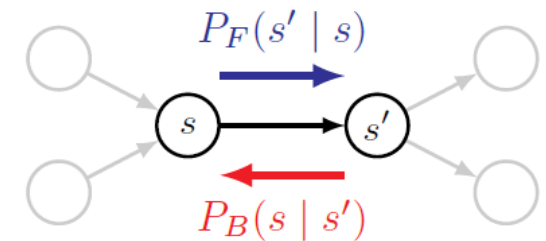
- **Detailed Balance (DB)**<sup>2</sup>: Forward flow = backward flow for each  $(s, s')$ .

- Target network: forward policy  $P_F(s'|s; \theta)$
- Auxiliary networks:
  - Backward policy  $P_B(s|s'; \theta)$
  - state flow function  $F(s; \theta)$ .
- DB loss

$$\ell_{DB}(s, s'; \theta) = \log \frac{F(s; \theta) P_F(s'|s; \theta)}{F(s'; \theta) P_B(s|s'; \theta)} \quad (F(x; \theta) = R(x))$$



Trajectory Balance



Detailed Balance

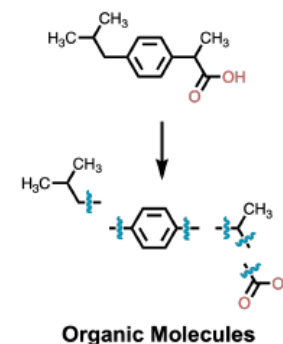
1. Bengio, Y., Lahlou, S., Deleu, T., Hu, E. J., Tiwari, M., & Bengio, E. (2023). Gflownet foundations. *Journal of Machine Learning Research*, 24(210), 1-55.  
2. Malkin, N., Jain, M., Bengio, E., Sun, C., & Bengio, Y. (2022). Trajectory balance: Improved credit assignment in GFlowNets. *Advances in Neural Information Processing Systems*, 35, 5955-5967.  
3. Deleu, T., Nishikawa-Toomey, M., Subramanian, J., Malkin, N., Charlin, L., & Bengio, Y. (2024). Joint bayesian inference of graphical structure and parameters with a single generative flow network. *Advances in Neural Information Processing Systems*, 36.

# Background – Generative Flow Networks (GFlowNets)

## ■ GFlowNet 연구 동향

### • Applications

- Discrete object를 다양하게 생성하는 것이 중요한 task.
  - Biological sequence<sup>1</sup>
  - Thread scheduling<sup>2</sup>
- Bayesian modeling / causal discovery<sup>3</sup>



Tyr-Pro-Trp-Phe

Tyr Pro Trp Phe

Protein Sequences

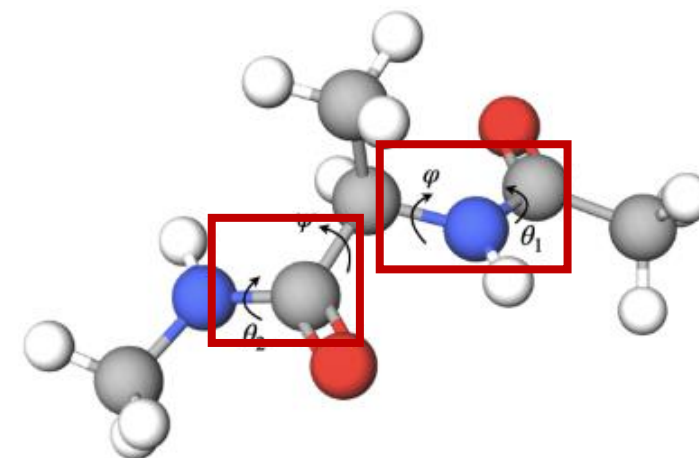
$\text{LiNi}_x\text{Mn}_y\text{Co}_z\text{O}_2$

Li xNi yMn zCo 2O

Inorganic Materials

### • Extension to continuous space<sup>4</sup>

- 더 다양한 데이터 및 task에 적용시키는 것이 목표.
  - Molecular structure + properties / annotations
  - Bayesian structure + parameter learning
  - Image generation



1. Bengio, E., Jain, M., Korablyov, M., Precup, D., & Bengio, Y. (2021). Flow network based generative models for non-iterative diverse candidate generation. *Advances in Neural Information Processing Systems*, 34, 27381-27394.  
2. Zhang, D. W., Rainone, C., Peschl, M., & Bondesan, R. (2022, September). Robust Scheduling with GFlowNets. In *The Eleventh International Conference on Learning Representations*.  
3. Deleu, T., Góis, A., Emezue, C., Rankawat, M., Lacoste-Julien, S., Bauer, S., & Bengio, Y. (2022, August). Bayesian structure learning with generative flow networks. In *Uncertainty in Artificial Intelligence* (pp. 518-528). PMLR.  
4. Lahlou, S., Deleu, T., Lemos, P., Zhang, D., Volokhova, A., Hernández-García, A., ... & Malkin, N. (2023, July). A theory of continuous generative flow networks. In *International Conference on Machine Learning* (pp. 18269-18300). PMLR.

# **02 Methods**

# Methods – MDP Design for GFlowNet

## ▪ GFlowNet 기반 CO를 위한 Markov Decision Process (MDP) 설계

- **Graph combinatorial optimization**

- $g = (V, E)$
- E.g., maximum independent set (MIS), maximum clique (MC).

- **State**

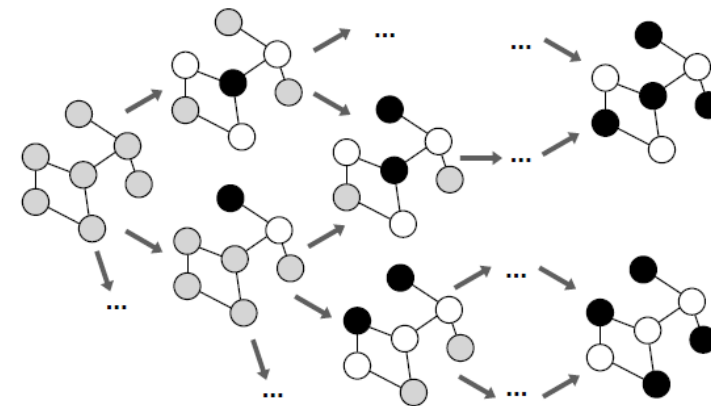
- $\bar{s} \triangleq \{(s^1, \dots, s^{|V|}) : s^d \in \{0, 1, \emptyset\}, d = 1, \dots, |V|\}$ .
- $s_0 = (\emptyset, \dots, \emptyset), x \in \mathcal{X} = \{0, 1\}^{|V|}$ .

- **Action:** choose one void vertex and add it ( $s^d \rightarrow 1$ ).

- **Transition:** feasibility 만족하도록 state 자동으로 update.

- **Reward:**  $R(x) = \exp(-C(x)/T)$

- $C(x)$ : CO의 cost function.
- $T \rightarrow \infty$ : uniform sampling /  $T \rightarrow 0$ : optimal solution sampling.



# Methods – Solving Training Inefficiency

## ▪ Issue 1: Training with long trajectory

### • Trajectory-based GFlowNet training

- 1) Forward process로 solution 생성하고, 이 때의 trajectory를 replay buffer에 저장.
- 2) Replay buffer로부터 하나의 trajectory 샘플링.
- 3) 해당 complete trajectory에 대해 DB or TB loss 계산 및 SGD.

- **Computational inefficiency:** Graph의 커지면서 loss 계산하기 위한 inference 횟수( $\propto$  trajectory 길이) 증가.

## ▪ Issue 2: Reward propagation / credit assignment

- GFlowNet의 유일한 학습 signal은 terminal state로부터 얻는 reward.
- **Reward propagation:** 해당 signal이 initial state까지 영향을 미치려면 오랜 시간 학습이 진행되어야 한다.
- **Credit assignment:** 해당 signal을 얻는 데에 어떤 action이 얼마만큼 기여했는지 알기 어렵다.

# Methods – Solving Training Inefficiency

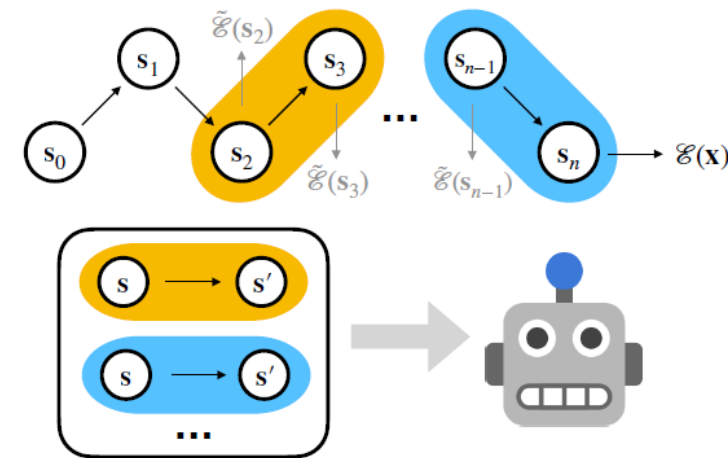
## ▪ Solutions

### • Transition-based GFlowNet training

- 1) Forward process로 solution 생성하고, 이 때의 transition들을 replay buffer에 저장.
- 2) Replay buffer로부터 B 개의 transition들을 샘플링하여 배치 구성.
- 3) 해당 배치에 대해 DB loss 계산 및 SGD.
  - 더 적은 GPU memory / training time.

### • Forward looking loss<sup>1</sup> with intermediate learning signal

- $\ell_{FL}(s, s'; \theta) = \log \frac{\tilde{R}(s)F(s; \theta)P_F(s'|s; \theta)}{\tilde{R}(s')F(s'; \theta)P_B(s|s'; \theta)}$
- $\tilde{R}(s)$ : intermediate state에서 정의된 reward function.
- 학습 수렴속도 더 빠르게.



---

#### Algorithm 1 GFlowNet training algorithm

**Require:** Graph dataset, GFlowNet model with parameters  $\theta$ , reward function.

**repeat**

Sample graph  $g$  from dataset;  
Sample  $\tau$  with the GFlowNet forward policy  $P_F(\cdot|\cdot; g, \theta)$ ;  
Randomly choose transitions from  $\tau$  to create buffer  $\mathcal{B}$ ;  
 $\Delta\theta \leftarrow \sum_{(s \rightarrow s') \in \mathcal{B}} \nabla_{\theta} \ell_{FL}(s, s'; g, \theta)$   
(as per Equation 6);  
Update  $\theta$  with some optimizer;

**until** some convergence condition

---

# Methods – Solving Training Inefficiency

## ▪ Effectiveness of solutions

### • Solution 1: Transition-based

- DB-traj  $\cong$  TB
- DB > DB-traj
- FL (ours) > FL-traj

### • Solution 2: Intermediate learning signal

- FL-traj > DB-traj
- FL (ours) > DB

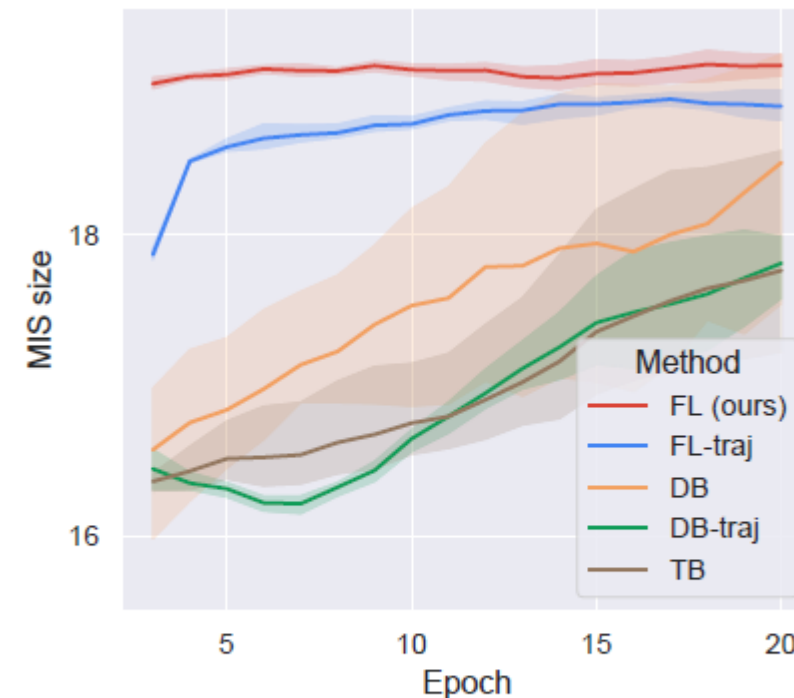


Figure 4: Comparison between different GFlowNet variants.

- **FL (ours)**: forward looking + transition sampling
- **FL-traj**: forward looking + trajectory sampling
- **DB-traj**: detailed balance + trajectory sampling
- **DB**: detailed balance + transition sampling
- **TB**: trajectory balance

# **03 Experiments**



# Experiments – Main

## ■ Main Experiments

### • Objectives

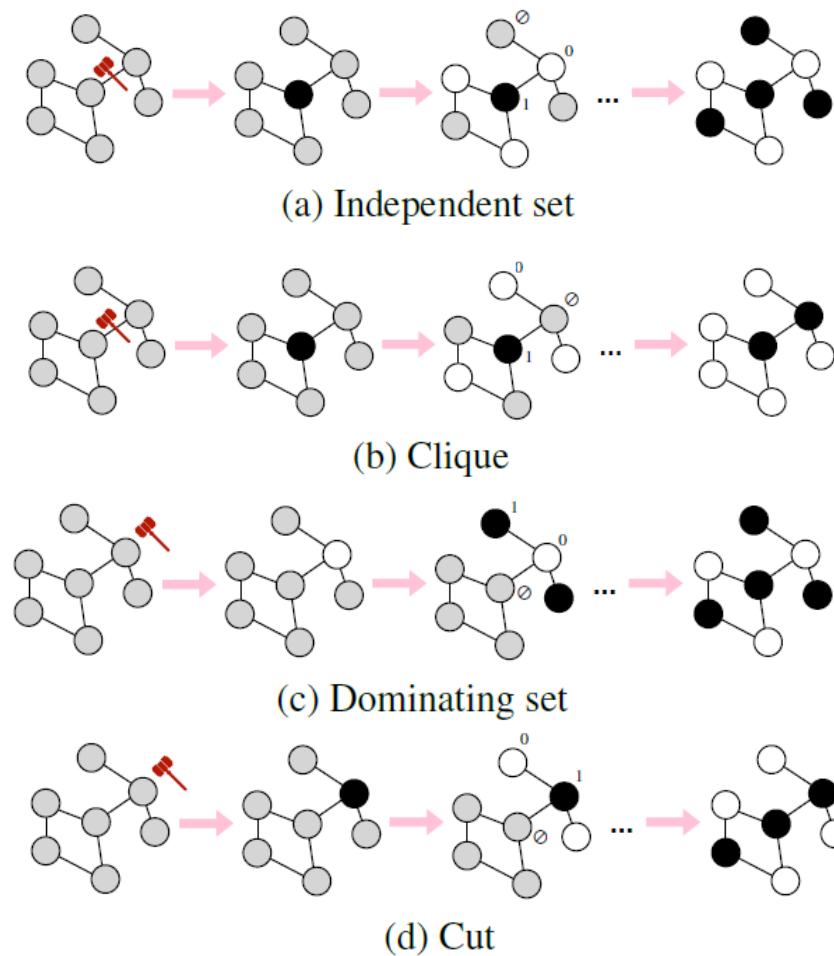
- **Effectiveness**: optimal solution(혹은 best result)과의 차이.
- **Efficiency**: inference time.

### • Tasks

- Maximum independent set (MIS) / Maximum clique (MC)
- Minimum dominating set (MDS) / Maximum cut (Mcut)

### • Datasets

- Synthetic data (RB graph<sup>1</sup> / BA graph<sup>2</sup>)
  - SMALL: 200~300 vertices.
  - LARGE: 800~1200 vertices.
- Realistic data (STALIB<sup>3</sup>)



1. Xu, K., & Li, W. (2000). Exact phase transitions in random constraint satisfaction problems. *Journal of Artificial Intelligence Research*, 12, 93-103.  
2. Barabási, A. L., & Albert, R. (1999). Emergence of scaling in random networks. *science*, 286(5439), 509-512.  
3. Hoos, H. H., & Stützle, T. (2000). SATLIB: An online resource for research on SAT. *Sat*, 2000, 283-292.

# Experiments – Main

## ▪ Experimental Details

### • Training details

- $P_F(s'|s; \theta)$ ,  $F(s; \theta)$ : 5 layers Graph isomorphism network (GIN)<sup>1</sup> with 256 hidden dim
- $P_B(s|s'; \theta)$ : uniform distribution (fixed)
- $T$  (@  $R(x) = \exp(-C(x)/T)$ ): 500
- **Training**: Adam(lr=0.001), 20 epochs, on-policy exploration for sampling.
- **Inference**: Sample 20 solutions & select the best.

1. Xu, K., Hu, W., Leskovec, J., & Jegelka, S. (2018). How powerful are graph neural networks?. *arXiv preprint arXiv:1810.00826*.

# Experiments – Main

---

- **Baselines**

- **For MIS problems**

- **GUROBI**: General-purpose mixed-integer program solver
    - **KAMIS**: MIS-specific solver
    - **PPO**: Reinforcement learning-based methods
    - **INTEL / DGL**: Supervised learning with tree search refinement

- **For non-MIS tasks**

- **GUROBI**: General-purpose mixed-integer program solver
    - **SDP**: Semi-definite programming baseline
    - **GREEDY**: Greedy annealing
    - **MFA**: Mean-field annealing
    - **ERDOS / ANNEAL**: Unsupervised learning based method

# Experiments – Main

## ▪ Results

### • MIS task

- GFlowNets strongly outperforms the previous neural solvers on all settings.
- For SATLIB, none of the SL baselines could achieve meaningful results within 10 hours.
- Although the RL-based solver (PPO) acquired meaningful results, it shows lower performance than the proposed model.

| METHOD | TYPE | SMALL        |        |         | LARGE        |        |         | SATLIB        |        |         |
|--------|------|--------------|--------|---------|--------------|--------|---------|---------------|--------|---------|
|        |      | SIZE ↑       | DROP ↓ | TIME ↓  | SIZE ↑       | DROP ↓ | TIME ↓  | SIZE ↑        | DROP ↓ | TIME ↓  |
| GUROBI | OR   | 19.98        | 0.01%  | 47:34   | 40.90        | 5.21%  | 2:10:26 | 425.95        | 0.00%  | 3:43:19 |
| KAMIS  | OR   | 20.10        | 0.00%  | 1:24:12 | 43.15        | 0.00%  | 2:03:36 | 425.96        | 0.00%  | 4:15:41 |
| PPO    | UL   | 19.01        | 5.42%  | 1:17    | 32.32        | 25.10% | 7:33    | 421.49        | 1.05%  | 13:12   |
| INTEL  | SL   | 18.47        | 8.11%  | 13:04   | 34.47        | 20.12% | 20:17   | —             | —      | —       |
| DGL    | SL   | 17.36        | 13.61% | 12:47   | 34.50        | 20.05% | 23:54   | —             | —      | —       |
| OURS   | UL   | <b>19.18</b> | 4.57%  | 0:32    | <b>37.48</b> | 13.14% | 4:22    | <b>423.54</b> | 0.57%  | 23:13   |

# Experiments – Main

## ■ Results

### • Non-MIS tasks

- GFlowNets outperformed the previous heuristic and UL-based solvers on SMALL & LARGE.

| METHOD | TYPE | MC           |        |        | MDS          |        |        | MCut          |        |        |
|--------|------|--------------|--------|--------|--------------|--------|--------|---------------|--------|--------|
|        |      | SIZE ↑       | DROP ↓ | TIME ↓ | SIZE ↓       | GAP ↓  | TIME ↓ | SIZE ↑        | DROP ↓ | TIME ↓ |
| GUROBI | OR   | 19.05        | 0.00%  | 1:55   | 27.89        | 0.00%  | 1:47   | 732.47        | 0.00%  | 13:04  |
| SDP    | OR   | —            | —      | —      | —            | —      | —      | 700.36        | 4.38%  | 35:47  |
| GREEDY | H    | 13.53        | 28.98% | 0:25   | 37.39        | 25.41% | 2:13   | 688.31        | 6.03%  | 0:13   |
| MFA    | H    | 14.82        | 22.15% | 0:27   | 36.36        | 23.29% | 2:56   | <b>704.03</b> | 3.88%  | 1:36   |
| ERDOS  | UL   | 12.02        | 36.90% | 0:41   | 30.68        | 9.09%  | 1:00   | 693.45        | 5.33%  | 0:46   |
| ANNEAL | UL   | 14.10        | 25.98% | 0:41   | 29.24        | 4.62%  | 1:01   | 696.73        | 4.88%  | 0:45   |
| OURS   | UL   | <b>16.24</b> | 14.75% | 0:42   | <b>28.61</b> | 2.52%  | 2:20   | <b>704.30</b> | 3.85%  | 2:57   |

*Max clique, min dominating set, max cut results on small graphs*

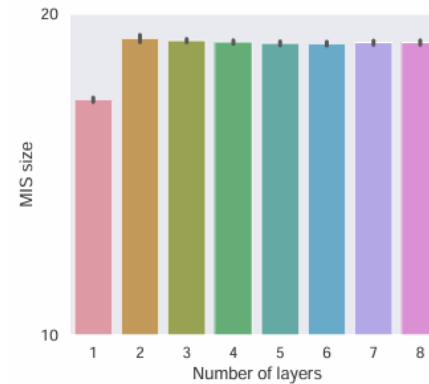
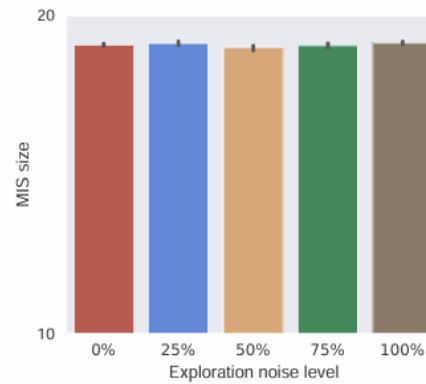
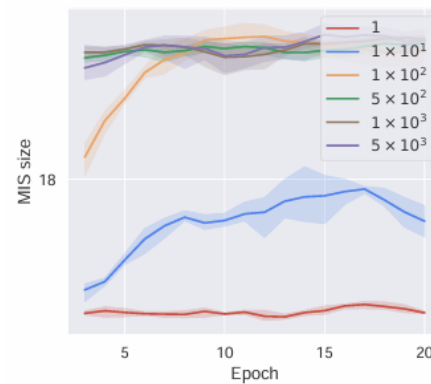
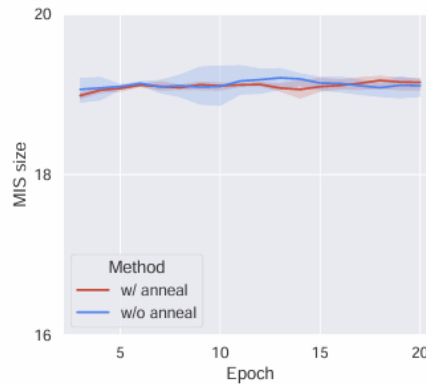
| METHOD | TYPE | MC           |        |        | MDS           |        |        | MCut           |        |          |
|--------|------|--------------|--------|--------|---------------|--------|--------|----------------|--------|----------|
|        |      | SIZE ↑       | DROP ↓ | TIME ↓ | SIZE ↓        | GAP ↓  | TIME ↓ | SIZE ↑         | DROP ↓ | TIME ↓   |
| GUROBI | OR   | 33.89        | 0.00%  | 16:40  | 103.80        | 0.00%  | 13:48  | 2915.29        | 0.00%  | 1:05:29  |
| SDP    | OR   | —            | —      | —      | —             | —      | —      | 2786.00        | 4.43%  | 10:00:00 |
| GREEDY | H    | 26.71        | 21.17% | 0:25   | 140.52        | 26.13% | 35:01  | 2761.06        | 5.29%  | 3:07     |
| MFA    | H    | 27.94        | 17.56% | 2:19   | 126.56        | 17.98% | 36:31  | 2833.86        | 2.79%  | 7:16     |
| ERDOS  | UL   | 25.43        | 24.96% | 2:16   | 116.76        | 11.10% | 3:56   | <b>2870.34</b> | 1.54%  | 2:49     |
| ANNEAL | UL   | 27.46        | 18.97% | 2:16   | 111.50        | 6.91%  | 3:55   | 2863.23        | 1.79%  | 2:48     |
| OURS   | UL   | <b>31.42</b> | 7.29%  | 4:50   | <b>110.28</b> | 5.88%  | 32:12  | 2864.61        | 1.74%  | 21:20    |

*Max clique, Min dominating set, and max cut results on large graphs*

# Experiments – Ablation Study

## ▪ Ablation Study

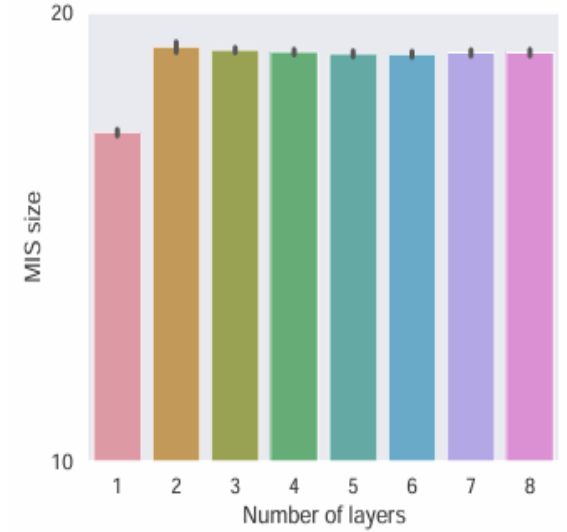
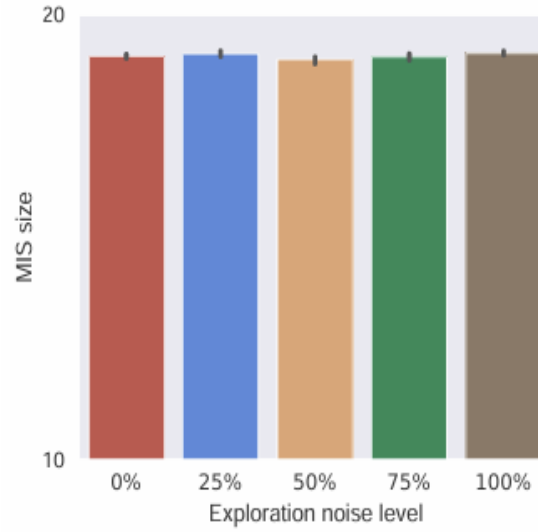
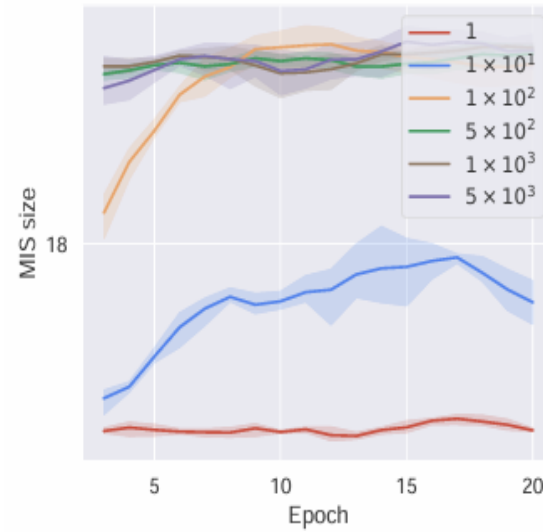
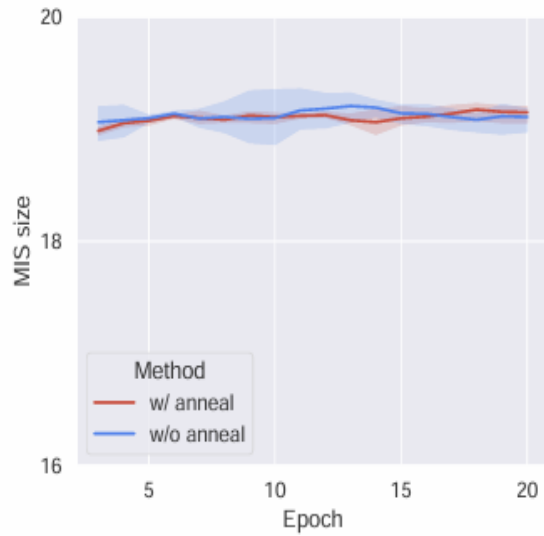
- **Objectives:** hyperparameter들에 대한 robustness 검증
- **Hyperparameters**
  - Temperature scale
  - Temperature annealing 여부: 초기 학습 시 더 높은  $T$ 로 설정.
  - Exploration strategy 적용 여부:  $(1 - \epsilon)P_F(s'|s; \theta) + \epsilon U(s')$
  - GIN layer 수



# Experiments – Ablation Study

## ■ Results

- 모두 robust하다는 결론.



# **04 Conclusion**



# Conclusion

---

## ▪ Contributions

- Proposed to apply the principled GFlowNet decision-making framework to CO tasks.
- Propose an efficient GFlowNet learning algorithm for
  - Enabling fast credit assignment for the GFlowNet agents
  - Handling long trajectories that emerge frequently in graph CO problems.
- The empirical advantage of GFlowNets is validated through experiments on different CO problems.

**End of Document**